

Part 1: SYSTEM ARCHITECTURE AND MEMORY MANAGEMENT

1.1. Memory Map

The program does not use a dynamic heap but rigidly maps memory regions within the `.data` segment.

A. Static Data

Label	Size	Address (Rel)	Detailed Description
buffer	256 bytes	0x10010000	Input Buffer: Stores the entire character string entered by the user before processing. A size of 256B is sufficient for the longest instruction line.
token	64 bytes	0x10010100	Processing Unit: Intermediate buffer. When the Parser slices the string, individual words (opcode, register, number) are copied here for analysis.
opcode_map	~300 bytes	...	Lookup Table: Database containing <code><Instruction Name String> - <Type ID></code> pairs. Used to identify instructions.

B. Hardware Interface (MMIO)

The program interacts with memory addresses mapped to simulated peripheral devices:

Device	Register	Address	Critical Bit	Function
Keyboard	CTRL	0xFFFF0000	Bit 0 (Ready)	Equals 1 when a new key is pressed.

Keyboard	DATA	0xFFFF0004	Low 8 bits	Contains the ASCII code of the character just received.
Display	CTRL	0xFFFF0008	Bit 0 (Ready)	Equals 1 when the screen is ready to display a new character.
Display	DATA	0xFFFF000C	Low 8 bits	Write ASCII code here to print to the screen.

Part 2: DATA I/O MECHANISM (LOW-LEVEL I/O DRIVER)

The program implements its own device "Driver" instead of using an operating system.

2.1. Polling Algorithm

Since Interrupts are not used, the CPU must continuously ask the device "Are you free?".

Logic Code:

Đoạn mã

wait_display:

```
li t0, DISPLAY_CTRL    # Load control address (0xFFFF0008)
lw t3, 0(t0)           # Read status
andi t3, t3, 0x1       # Mask to get the last bit (Ready Bit)
beqz t3, wait_display  # If bit = 0, loop back and wait
li t0, DISPLAY_DATA    # If bit = 1, device is ready
sw t2, 0(t0)           # Send data (t2) to screen
```

Analysis: This is a **synchronous** mechanism. The CPU is blocked here until the peripheral responds.

2.2. Backspace Handling (User Experience)

When ASCII 8 (Backspace) or 127 (Delete) is detected, the program executes a complex sequence of actions to visually trick the user:

- **Memory Logic:** Decrease the **buffer** pointer **s0** by 1 unit (so the next input will overwrite the incorrect character).
- **Display Logic:** Send the byte sequence **8, 32, 8** to the screen.
 - **8:** Move screen cursor to the left (over the incorrect character).
 - **32 (Space):** Print a space (erase the image of the character).
 - **8:** Move the cursor back to the erased position (ready to print a new character).

Part 3: CORE PARSING ALGORITHMS (CORE PARSERS)

This is the "brain" of the program, where string logic is processed.

3.1. Tokenizer - Function `get_next_token`

Transforms an instruction line like `add t0, t1, 10` into separate tokens: `add`, `t0`, `t1`, `10`.

- **Input:** Pointer `s1` (current position in buffer).
- **Delimiters:** Space (32), Tab (9), Comma (44), Newline (10).
- **Algorithm:**
 1. **Skip Whitespace:** Loop to ignore all leading delimiter characters.
 2. **Copy Token:** Copy non-delimiter characters into the `token` memory.
 3. **Null-Terminate:** Add a `\0` character to the end of the token to create a standard C-String.
 4. **Update:** Move pointer `s1` to the next position to prepare for the subsequent call.

3.2. Number Parser - Function `parse_immediate`

Converts a character string (String) to a 32-bit Integer.

- **Supported Formats:**
 - **Decimal:** `123`, `-45`.
 - **Hexadecimal:** `0xFF`, `0x1A`.
- **Decimal Algorithm:**
 $\text{Value} = (\text{Value} * 10) + (\text{Char} - '0')$
- **Hex Algorithm:**
 $\text{Value} = (\text{Value} \ll 4) + \text{Hex_Digit}$
(Left shift 4 bits is equivalent to multiplying by 16).
- **Sign Handling:** Uses flag register `t6`. If `-` is detected at the start, `t6 = 1`. At the end of the function, if `t6 == 1`, execute `neg` (2's complement) on the result.

3.3. Register Validator - Function `check_is_register`

Checks the validity of register operands via two flows:

- **Flow 1: ABI Names (Application Binary Interface):** Compares the token string with the string database: `zero`, `ra`, `sp`, `gp`, `tp`, `t0...t6`, `s0...s11`, `a0...a7`. Uses a byte-by-byte comparison loop (`lb`, `bne`).
- **Flow 2: Hardware Names (x-register):**
 - Checks if the first character is `x`.
 - Parses the following number into an integer.
 - **Range Check:** Checks if the number satisfies $0 \leq N \leq 31$.

Part 4: INSTRUCTION DISPATCHER

The program uses a mapping table (`opcode_map`) to determine validation logic.

4.1. Opcode Map Structure

The database is organized as an array of Structs:

Đoạn mã

opcode_map:

```
.word op_lw, 1    # String "lw" -> ID 1
.word op_add, 4   # String "add" -> ID 4
.word op_beq, 5   # String "beq" -> ID 5
# ...
.word 0, 0        # Sentinel (Guard to signal end of table)
```

4.2. Instruction Type ID Table

The program defines internal IDs to reuse validation code:

- **ID 1 (Type Load):** Check Reg, Address.
- **ID 10 (Type Store):** Check Reg, Address (Logic same as Load).
- **ID 2 (Type U):** Check Reg, Imm20.
- **ID 3 (Type I):** Check Reg, Reg, Imm12.
- **ID 7 (Type Shift):** Check Reg, Reg, Imm5.
- **ID 4 (Type R):** Check Reg, Reg, Reg.
- **ID 5 (Type B):** Check Reg, Reg, Label.
- **ID 6 (Type J):** Check Reg, Label.

Part 5: DETAILED VALIDATION LOGIC

This is the most critical part, describing how the program determines correctness for each instruction type.

5.1. R-Type Processing (add, sub, or...)

- **Syntax:** OP rd, rs1, rs2
- **Process:**
 1. Call `check_reg_op1`: Validate rd.
 2. Call `check_reg_op2`: Validate rs1.
 3. Call `check_reg_op3`: Validate rs2.
- **Error Example:** add t0, 1, t2 -> Error at Operand 2 because 1 is not a register.

5.2. Load/Store Processing (lw, sw...)

- **Syntax:** OP reg, offset(base)
- **Process:**
 1. Call `check_reg_op1`.
 2. Call `check_address_op2`:
 - Split before (: Call `check_is_12bit_imm`. Offset must be in range [-2048, 2047].
 - Split inside (): Call `check_is_register`. Base must be a valid register.
- **Error Example:** lw t0, 4000(sp) -> Error Offset because 4000 > 2047.

5.3. I-Type Processing (**addi**, **andi**...)

- **Syntax:** **OP** **rd**, **rs1**, **imm**
- **Process:**
 1. Check the first 2 registers.
 2. Call **check_12bit_imm_op3**.
- **Note:** The **jalr** (Jump And Link Register) instruction structure is also I-Type, so it is categorized here.

5.4. Shift Processing (**slli**, **srai**...)

- **Syntax:** **OP** **rd**, **rs1**, **shamt**
- **Process:**
 1. Check the first 2 registers.
 2. Call **check_5bit_imm_op3**: Check Shift Amount.
- **Condition:** $0 \leq \text{shamt} \leq 31$.
- **Reason:** RV32 architecture has only 32 bits; shifting beyond 31 positions is invalid.

5.5. Branch Processing (**beq**, **bne**...)

- **Syntax:** **OP** **rs1**, **rs2**, **label**
- **Process:**
 1. Check the 2 comparison registers.
 2. Call **check_label_op3**: Check jump label.
- **Label Rules:** Must start with a letter or **_**. Must not contain special characters.
- **Validation:** The label name must not conflict with reserved keywords (Opcode).
Example: **beq t0, t1, lw** is invalid syntax.

Part 6: ERROR HANDLING

The program applies the **Fail-Fast** principle (Stop immediately upon encountering an error).

6.1. Error Codes

Error messages are predefined in **.data**:

- **msg_err_op**: "Error: Invalid or unsupported opcode."
- **msg_err_op1**: "Error: Invalid Operand 1."
- **msg_err_op2**: "Error: Invalid Operand 2."
- **msg_err_op3**: "Error: Invalid Operand 3."

6.2. Control Flow

Each Checker function returns the result in register **a0**:

- **a0 = 1**: Valid.
- **a0 = 0**: Invalid.

After each checker call, the program checks **a0** immediately:

Đoạn mã

jal check_is_register

beqz a0, e_op1 # If a0 == 0, jump immediately to error label e_op1